

Modélisation et vérification formelle d'un système bancaire distribué

Akka, Scala, réseaux de Pétri et LTL

CY Tech - ING2 - PRF | Projet 2026

Groupe 5 : Souleim, Mehdi, Abbes, Nahel

Repository : <https://github.com/souleim95/Systeme-Bancaire-Distribue.git>

Résumé

Ce projet modélise et vérifie une application bancaire distribuée manipulant des comptes, dépôts, retraits et virements. L'application opérationnelle repose sur Akka Typed et Scala. Les comportements critiques sont abstraits dans un réseau de Pétri maison afin d'explorer l'espace d'états et de vérifier des propriétés de sûreté et de vivacité.

Le résultat couvre les exigences du cahier des charges : état de l'art, architecture acteur, réseau de Pétri, analyse structurelle, invariants métier, model checking LTL, tests unitaires, interface de démonstration et comparaison Akka/Pétri.

1. Introduction

Les systèmes bancaires distribués doivent traiter des opérations concurrentes tout en préservant des garanties fortes : absence de perte d'argent, refus des opérations invalides, terminaison explicite des transactions et absence de blocage global.

- application Akka/Scala pour simuler le comportement réel ;
- réseau de Pétri pour explorer les états critiques ;
- LTL pour exprimer et vérifier des propriétés temporelles.

2. État de l'art

La vérification formelle représente mathématiquement le comportement d'un système puis vérifie que certaines propriétés sont satisfaites. Elle complète les tests en explorant des chemins difficiles à reproduire manuellement.

Notion	Définition	Exemple bancaire
Sûreté	Quelque chose de mauvais n'arrive jamais.	Un compte ne devient jamais négatif.
Vivacité	Quelque chose de bon finit par arriver.	Un virement finit par être confirmé ou annulé.
Deadlock	Aucune progression n'est possible.	Aucun état atteint ne doit bloquer tout le système.

Les réseaux de Pétri représentent naturellement la concurrence, les conflits de ressources et l'évolution d'un système à travers les marquages. LTL ajoute une couche de propriétés temporelles comme G phi, F phi, X phi ou phi U psi.

3. Architecture Akka/Scala

Composant	Responsabilités
Banque	Crée les comptes, route les opérations, agrège les soldes, arrête les comptes fermés.
Compte	Porte l'état d'un compte : solde, historique, statut actif, virements en attente.
ProtocoleBancaire	Définit les commandes et réponses échangées entre acteurs.
FrontServer	Expose une interface web locale de démonstration.

Flux de messages

Client -> Banque -> Compte -> Banque -> Client

Le virement est le flux critique : la source réserve immédiatement le montant, contacte la destination, puis confirme ou rembourse selon la réponse. Cette logique protège contre un double débit concurrent.

Un compte bancaire ne doit jamais avoir un solde négatif.

4. Modèle formel par réseau de Pétri

Fichier	Rôle
PetriNet.scala	Structures Place, Transition, Arc, Marking, franchissement et graphe de réachabilité.
BankingPetriNet.scala	Réseaux bancaires : compte simple, virement, réseau complet multi-comptes.
PropertyChecker.scala	Vérification deadlock, vivacité, bornitude, réversibilité et invariants.
LTL.scala	Parseur, évaluateur et model checker LTL.
Simulator.scala	Simulation interactive, aléatoire, par trace et par séquence.

Réseau	Places	Transitions	Arcs	États atteignables
Compte simple	4	3	6	2
Virement deux comptes	7	4	22	3
Réseau complet trois comptes	15	21	96	216

5. Vérification des propriétés

- absence de deadlock : aucun marquage atteignable ne doit être terminal ;
- vivacité : les transitions critiques peuvent redevenir exécutables ;
- bornitude : le nombre de jetons reste borné ;
- marquages non négatifs : cohérence avec l'invariant métier.

Formule LTL	Signification
G enabled	Il existe toujours au moins une transition disponible.
G !deadlock	Aucun état atteignable n'est un deadlock.
G (locked -> F available)	Un verrou finit par être libéré.
G (transferInitiated -> F completed)	Un virement initié finit par atteindre une issue explicite.

6. Simulation et comparaison

Scénario Akka de référence :

- création de ACC-001 = 1000 EUR, ACC-002 = 500 EUR, ACC-003 = 750 EUR ;
- dépôt de 200 EUR sur ACC-001 ;
- retrait de 100 EUR sur ACC-002 ;
- virement de 300 EUR de ACC-001 vers ACC-002.

Compte	Solde final attendu
ACC-001	900 EUR
ACC-002	700 EUR

Compte	Solde final attendu
ACC-003	750 EUR

Akka montre l'exécution réelle avec acteurs, messages et historiques. Pétri vérifie l'abstraction formelle : disponibilité, verrouillage, transitions critiques, états atteignables et absence de blocage.

7. Interface de démonstration

```
sbt "runMain banque.FrontServer 8080"
http://localhost:8080
```

- créer, fermer et consulter des comptes ;
- effectuer dépôts, retraits et virements ;
- afficher le total banque et les historiques ;
- relancer l'analyse Pétri ;
- tester des formules LTL comme $G \text{ enabled}$ et $G \text{ !deadlock}$.

8. Tests et validation

Suite	Couverture
CompteSpec	Dépôt, retrait, solde insuffisant, fermeture, historiques.
BanqueSpec	Création, routage, virements, agrégation des soldes.
PetriNetBankingTests	Création des réseaux, transitions, graphe de réachabilité, invariants.
LTLTests	Parsing, opérateurs temporels, formules invalides, model checking.

```
sbt clean test
sbt "runMain banque.PetriNetIntegrationDemo"
sbt "runMain banque.LTLIntegrationExample"
```

9. Traçabilité avec le cahier des charges

Attendu	Couverture
Bibliographie	docs/etat_de_l_art.md et références du rapport.
Modèle Akka/Scala	src/main/scala/banque.
Réseau de Pétri	src/main/scala/petri/BankingPetriNet.scala.
Rapport de vérification	docs/rapport_verification.md et ce rapport final.
Simulation comparée	PetriNetIntegrationDemo.scala et interface web.
Lien GitHub	README.md et page de garde.

10. Limites et perspectives

- le modèle de Pétri abstrait les soldes par une validité plutôt que par toutes les valeurs monétaires possibles ;

- l'espace d'états augmente rapidement avec le nombre de comptes ;
- les contre-exemples LTL pourraient être visualisés dans l'interface ;
- extensions possibles : PNML, réseaux temporisés, CTL, optimisations BDD.

Conclusion

Le projet aboutit à une application bancaire distribuée complète, vérifiée par deux approches complémentaires. Akka/Scala fournit une simulation réaliste et testable, tandis que le réseau de Pétri et LTL fournissent une analyse formelle des comportements critiques.

Les propriétés principales sont couvertes : absence de deadlock, bornitude, vivacité, marquages non négatifs et cohérence des virements.

Bibliographie

- Christel Baier, Joost-Pieter Katoen, Principles of Model Checking, MIT Press, 2008.
- Wolfgang Reisig, Understanding Petri Nets, Springer, 2013.
- E. M. Clarke, O. Grumberg, D. Peled, Model Checking, MIT Press, 1999.
- Leslie Lamport, Proving the Correctness of Multiprocess Programs, IEEE TSE, 1977.
- Amir Pnueli, The Temporal Logic of Programs, FOCS, 1977.
- Documentation Akka Typed et documentation Scala.